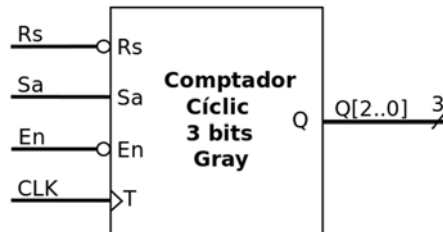


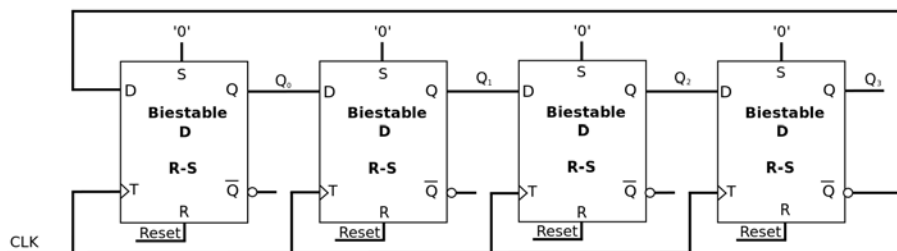
Problema 1 (30 minuts, 3 punts)

- a) Implementa un comptador cíclic de 3 bits amb codificació Gray, el qual posseeix com a senyals de control senyal d'Enable (EN), senyal de Reset síncron (Rs) i senyal de Set asíncron (Sa).



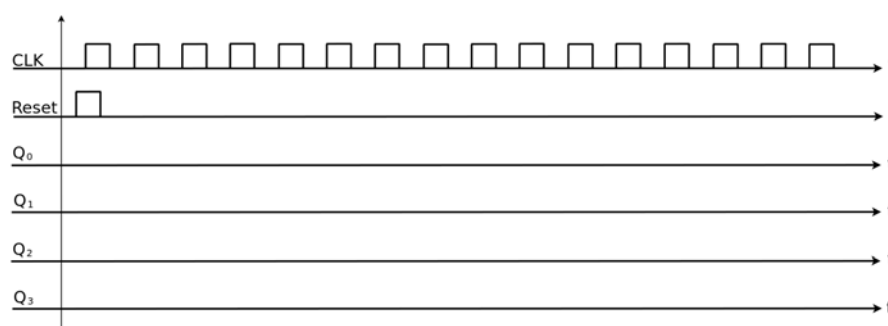
Descripció dels senyals de control:

- **En:** Senyal d'Enable, actiu per lògica negativa. És el més prioritari de tots els senyals síncrons.
 - **Rs:** Senyal de Reset síncron, actiu per lògica negativa.
 - **Sa:** Senyal de Set asíncron, actiu per lògica positiva. És el més prioritari de tots els senyals de control.
- b) Ens han facilitat el següent *Fed Shift Register* i aplicant enginyeria inversa hem de descobrir quin és el seu funcionament.



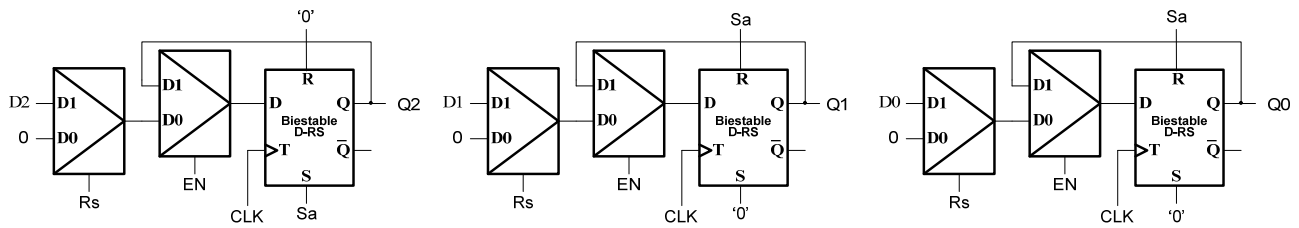
Per fer-ho s'ha introduït a les entrades de CLK i Reset els senyals que es veuen a continuació, de forma que es pugui analitzar el seu comportament a la sortida.

- 1) Obté el diagrama de temps següent per determinar quina és la seqüència que el *Fed Shift Register* genera.
- 2) Quina funcionalitat posseeix el dispositiu que ens han facilitat?



Solució:

a)



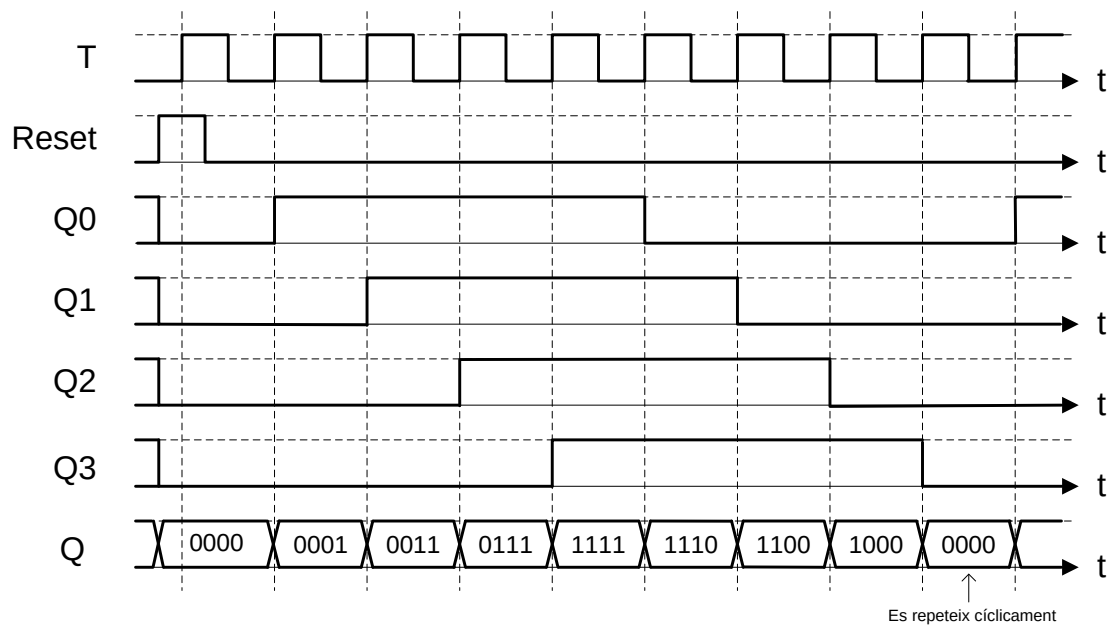
Q_2	Q_1	Q_0	D_2	D_1	D_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

$$D_2 = \overline{Q_2} \cdot Q_1 \cdot \overline{Q_0} + Q_2 \cdot Q_1 \cdot \overline{Q_0} + Q_2 \cdot Q_1 \cdot Q_0 + Q_2 \cdot \overline{Q_1} \cdot Q_0$$

$$D_1 = \overline{Q_2} \cdot \overline{Q_1} \cdot Q_0 + \overline{Q_2} \cdot Q_1 \cdot Q_0 + \overline{Q_2} \cdot Q_1 \cdot \overline{Q_0} + Q_2 \cdot Q_1 \cdot \overline{Q_0}$$

$$D_0 = \overline{Q_2} \cdot \overline{Q_1} \cdot \overline{Q_0} + \overline{Q_2} \cdot \overline{Q_1} \cdot Q_0 + Q_2 \cdot Q_1 \cdot \overline{Q_0} + Q_2 \cdot Q_1 \cdot Q_0$$

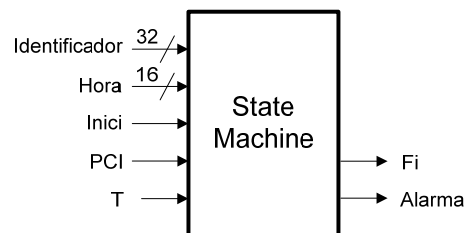
b)



És un comptador Johnson de 4 bits cíclic amb reset asíncron.

Problema 2 (55 minuts, 4 punts)

Empreses Wayne ha rebut un nou projecte per part del seu propietari, Bruce Wayne. Es tracta d'una modificació d'un control d'accés per a una porta de seguretat que hi ha instal·lada a la sala de mapes de la seva mansió i que en quedi constància de tots els intents que algú en pugui fer per accedir-hi. A la sala de mapes tan sols s'hi pot accedir entre les 8 del matí i les 10 del vespre.



Entrades:

- El senyal **Inici** indica que ha de començar un nou procés d'accés. L'usuari mantindrà actiu ('1') aquest senyal mentre el sistema estigui operant.
- **T** serà el senyal de sincronisme del sistema.
- **PCI** és un senyal d'inicialització que situa al sistema al seu estat inicial quan s'engega.
- **Identificador** és un senyal de 32 bits que conté l'identificador de la lectura de l'ull d'un lector de retina instal·lat a l'entrada de l'accés que es pretén controlar.
- **Hora** és un senyal de 16 bits que indicarà l'hora a la que s'intenta un accés.

Sortides:

- El senyal **Fi** és un únic bit que informa que el sistema ha acabat de processar totes les dades o que s'ha produït un error.
- El senyal **Alarma** valdrà '1' quan es produeixi un error en el sistema i que, per tant, hi ha un intrús que intenta accedir a la zona controlada.

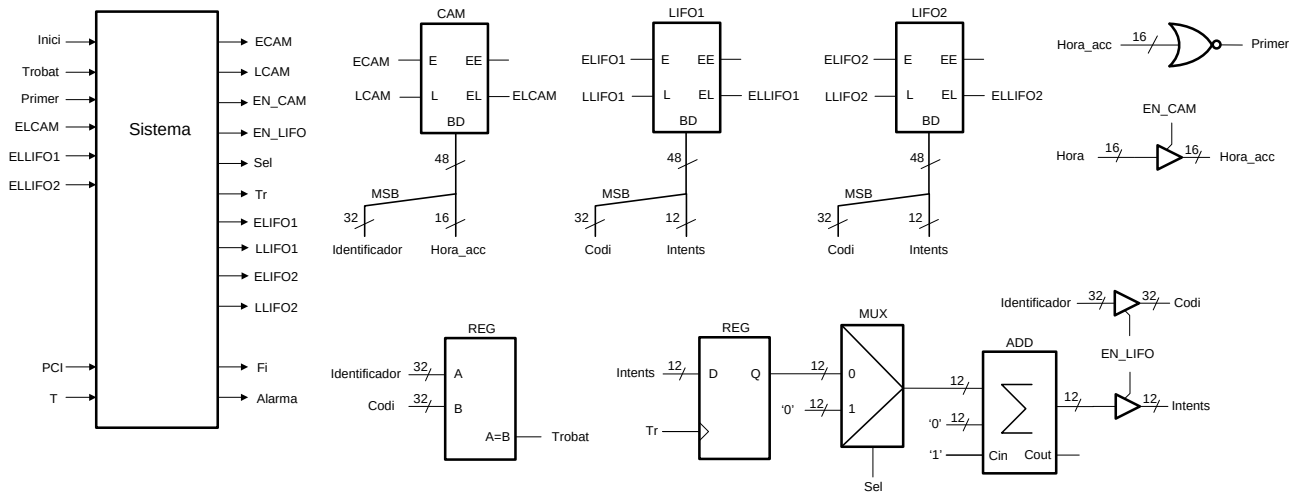
La base de dades del sistema de seguretat és dins d'una memòria CAM que fa servir com a índex el codi identificador de tota persona que tingui accés i que serà confidencial, és a dir que no serà rellevant per al desenvolupament de la part que ens han encarregat. Cal tenir en compte que cada persona solament pot accedir una única vegada a les instal·lacions. Per aquest motiu, la informació emmagatzemada a la CAM per a cada codi identificador és una paraula de 48 bits: el codi identificador de 32 bits com a índex, i l'hora del primer accés d'aquesta persona (16 bits). Quan es posa en marxa el sistema al principi del dia, la CAM conté els identificadors de totes les persones amb accés a la sala, i la resta d'informació tot a zeros.

El sistema disposa també d'una memòria LIFO on s'anirà emmagatzemant el codi identificador de totes les persones (autoritzades o no) que intentin accedir a la sala i el nombre total d'intents d'accés que han realitzat.

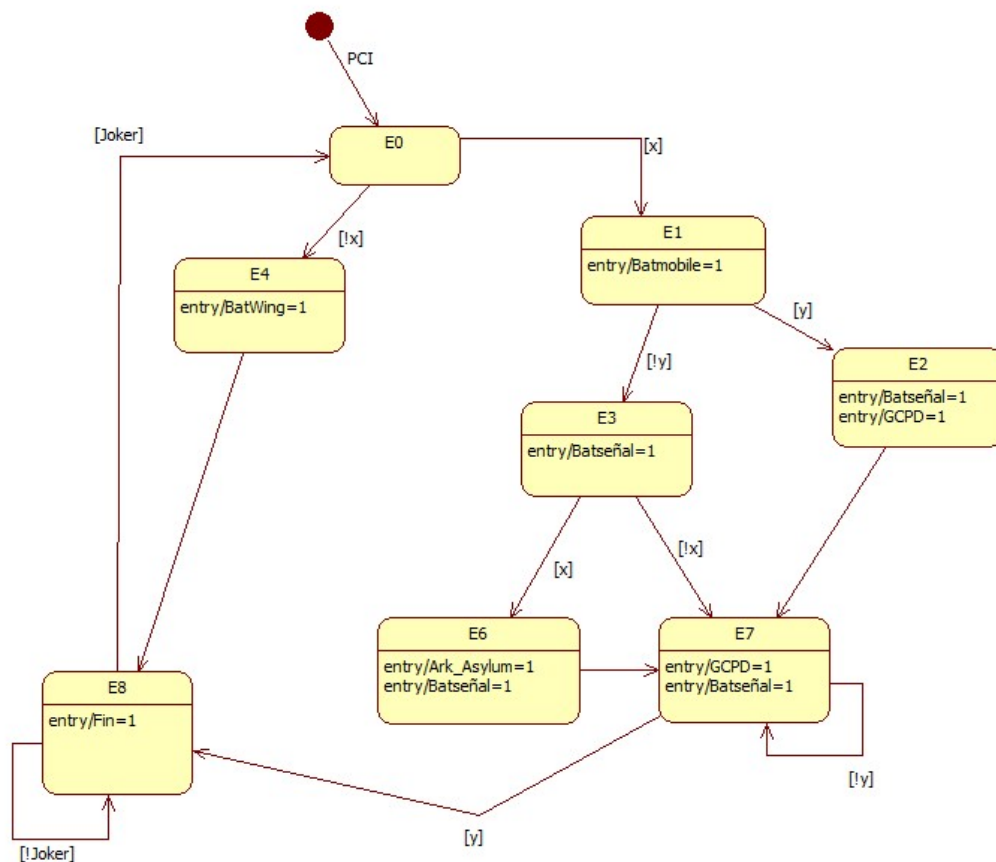
Un cop s'indiqui al sistema que comença un procés d'accés, activant el senyal **Inici**, el sistema llegirà les dades del senyal **Identificador** i accedirà a la CAM. Si la persona està autoritzada a entrar i és el seu primer intent d'accés es guardarà el codi d'accés i s'actualitzarà el nombre d'intents d'accés (en aquest cas, 1) a la memòria LIFO, s'actualitzarà la CAM amb les dades pertinents i s'activarà el senyal **Fi** durant un període del senyal de sincronisme (**T**) i el sistema tornarà a quedar a l'espera d'un nou accés. En cas que la persona no tingui accés o que ja hagi accedit prèviament al sistema, el sistema actualitzarà a la memòria LIFO el nombre d'intents d'accés, activarà el senyal **Alarma** durant un període complet i tornarà a l'estat de repòs esperant un nou accés.

Ens garanteixen que la LIFO estarà buida d'inici i que tindrà espai suficient per a emmagatzemar la totalitat de les dades que s'hi vulguin introduir.

- a) Empreses Wayne ha elaborat un hardware que es presenta a continuació i el que han demanat als enginyers de La Salle és el desenvolupament solament de la màquina d'estats que governa el Sistema Seqüencial Síncron. Dissenya el diagrama d'estats d'aquesta màquina d'estats.

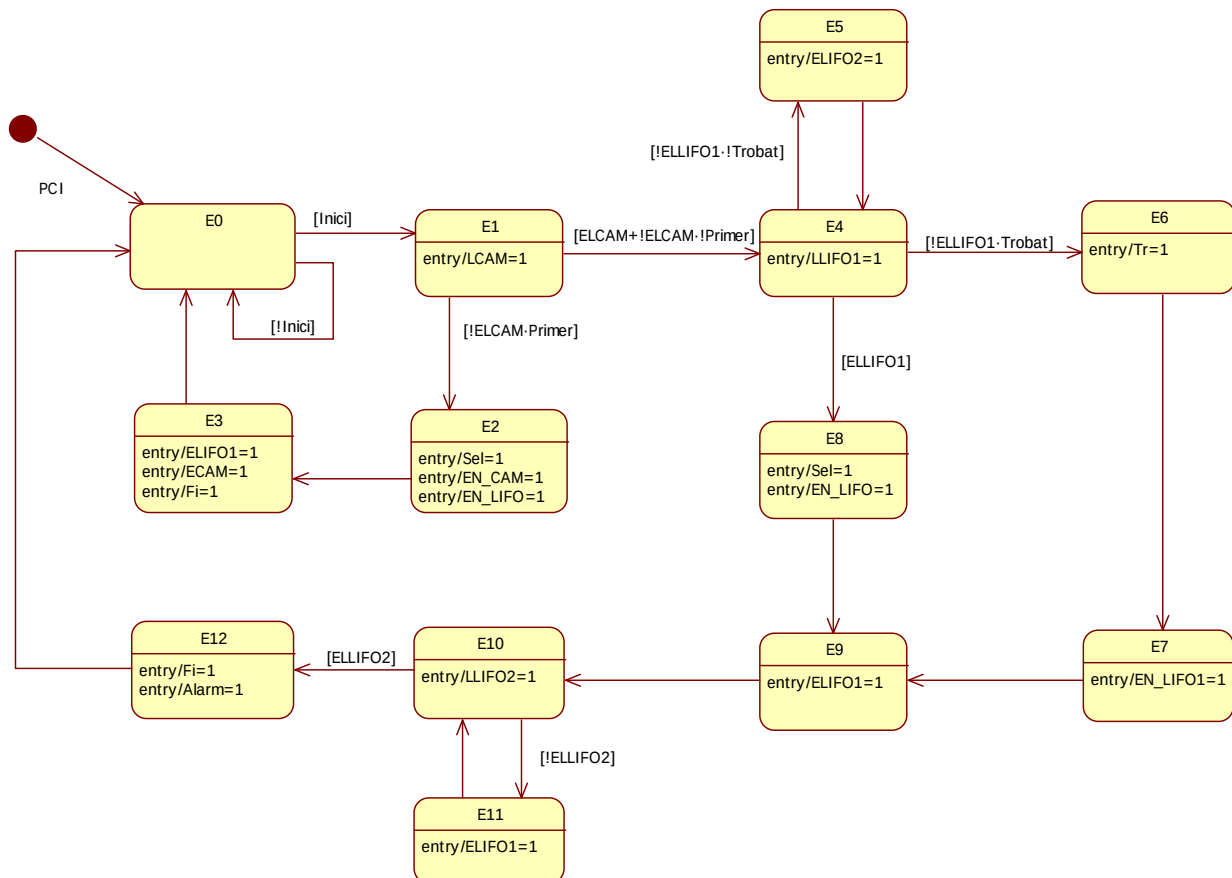


- b) Donat el següent diagrama d'estats, que no guarda relació amb el problema anterior, implementa-ho pel mètode 1:n.

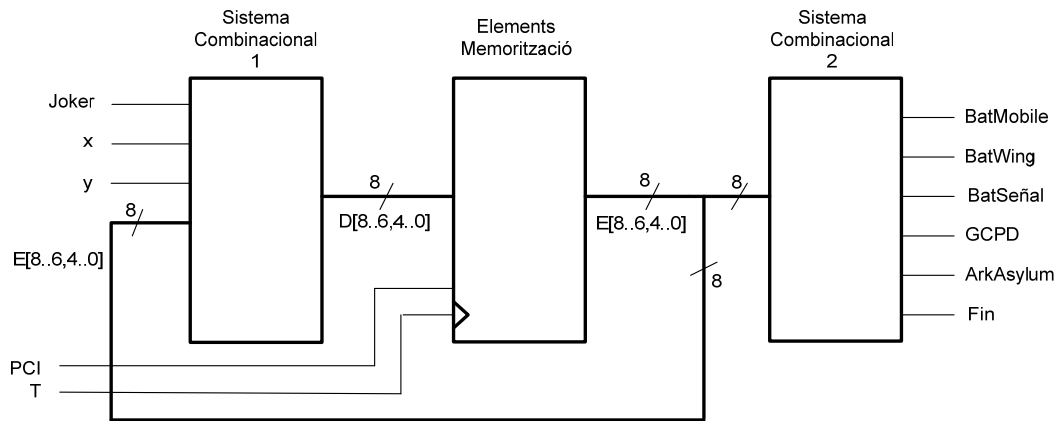


Solució:

- a) Quan s'activa Inici hem de llegir de la CAM i es poden donar dos situacions:
1. La persona està autoritzada i és el seu primer intent d'accés ([!ELCAM·Primer]). En aquest cas escrivim l'hora d'accés a la CAM i l'identificador i el nombre d'intents (que serà 1) a la LIFO1 i activar Fi (E2-E3).
 2. La persona està autoritzada però no és el seu primer intent d'accés, o no està autoritzada ([!ELCAM·!Primer+ELCAM]). En aquest cas no hem de fer res a la CAM però hem d'actualitzar el nombre d'intents a la LIFO1. Per fer-ho, hem d'anar llegint les dades de la LIFO1 i, mentre no trobem el codi de la persona (Trobat) hem d'anar guardant les dades llegides a una segona memòria (LIFO2) per què no es perdin. Del bucle E4-E5 (llegir de la LIFO1 i escriure a la LIFO2) es pot sortir per dos motius:
 - i. Llegim totes les dades de la LIFO1 sense trobar el codi de l'usuari ([ELLIFO1]). Llavors hem de guardar l'identificador i el nombre d'intents (igual a 1) a la LIFO1 (E8-E9).
 - ii. Trobem el codi de l'usuari (!ELLIFO1·Trobat]). Llavors guardem el nombre d'intents, sumem 1 i tornem a escriure a la LIFO1 l'identificador i el nombre d'intents incrementat (E6-E7-E9).
- Abans de finalitzar hem de restituir totes les dades de la LIFO2 a la LIFO 1 (bucle E10-E11) i finalitzar activant Alarm (E12).

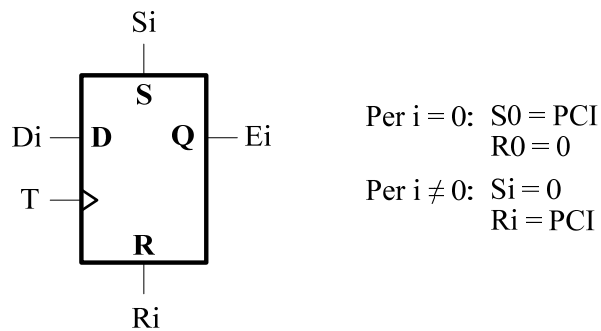


b) L'esquema general de la implementació serà el següent:



Bloc d'elements de memorització:

Estarà format per 8 biestables ($i = 0, 1, 2, 3, 4, 6, 7, 8$) amb el següent esquema d'interconnexions:



Sistema Combinacional 1 (proper estat):

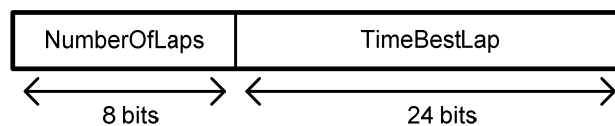
$$\begin{aligned} D_0 &= E_8 \cdot \text{Joker} \\ D_1 &= E_0 \cdot x \\ D_2 &= E_1 \cdot y \\ D_3 &= E_1 \cdot \bar{y} \\ D_4 &= E_0 \cdot \bar{x} \\ D_6 &= E_3 \cdot x \\ D_7 &= E_2 + E_3 \cdot \bar{x} + E_6 + E_7 \cdot \bar{y} \\ D_8 &= E_4 + E_7 \cdot y + E_8 \cdot \overline{\text{Joker}} \end{aligned}$$

Sistema Combinacional 2 (sortides):

$$\begin{aligned} \text{BatMobile} &= E_1 \\ \text{BatWing} &= E_4 \\ \text{BatSeñal} &= E_2 + E_3 + E_6 + E_7 \\ \text{GCPD} &= E_2 + E_7 \\ \text{ArkAsylum} &= E_6 \\ \text{Fin} &= E_8 \end{aligned}$$

Problema 3 (50 minuts, 3 punts)

La FIA (Federació Internacional d'Automobilisme) ens ha demanat dissenyar un sistema que enregistri els primers classificats i els millors temps per volta a diferents carreres dels campionats que organitzen. La informació dels competidors s'anirà actualitzant al llarg de la carrera en una memòria RAM que fa servir com a adreça un codi identificatiu de cada cotxe (CarCode) de 16 bits, i la informació que cal anar actualitzant és doble: d'una banda el nombre de voltes completades per cada cotxe (NumberOfLaps), codificada en binari natural amb 8 bits, i d'altra, el temps de la millor volta realitzada per aquell cotxe (TimeBestLap), en milisegons, codificat en binari natural amb 24 bits. Cada cop que un cotxe completa una volta, arriba al sistema la informació del temps realitzat pel cotxe en aquella volta, i el sistema ha d'actualitzar a la RAM el nombre de voltes completades i, si ha millorat el temps de volta, el nou millor temps de volta. L'estructura d'una paraula a la RAM és doncs:



Inicialment, la RAM es subministra tota plena de zeros, tal com es mostra a la part esquerra de la figura següent. Al cap d'una estona, un cop completades unes quantes voltes de carrera, el contingut de la RAM podria ser el que es mostra a la part dreta de la figura.

FFFFh	00000000	000000h
FFFEh	00000000	000000h
	...	...
0001h	00000000	000000h
0000h	00000000	000000h

FFFFh	00000101	0186B3h
FFFEh	00000000	000000h
	...	...
0001h	00000110	018A2Eh
0000h	00000000	000000h

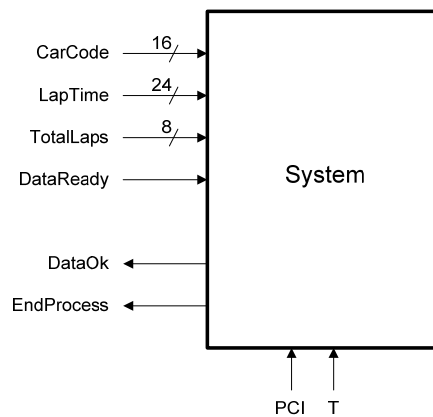
A més, en finalitzar la carrera, el sistema ha de subministrar la classificació dels primers 10 cotxes, indicant el codi identificatiu del cotxe (CarCode), la posició a la carrera (Position) codificada en binari natural amb 4 bits, i el temps de la millor volta de cada cotxe (TimeBestLap). Tota aquesta informació s'ha de subministrar emmagatzemada a una memòria FIFO de 100 paraules de capacitat, amb una estructura de paraula com la que es mostra a continuació:



En finalitzar la carrera, la informació emmagatzemada a la FIFO podria ser, per exemple, com la que es mostra a la figura següent:

...	...	...
1010101010101010	1010	018708h
1111111100000000	1001	018712h
...	...	...
0000111111110000	0010	018632h
0000111111111111	0001	0186F2h

El diagrama de blocs del sistema es mostra a la figura següent.



Entrades:

- **CarCode:** codi identificatiu del cotxe, de 16 bits. Roman estable mentre estigui actiu el senyal DataReady.
- **LapTime:** temps de la darrera volta del cotxe indicat a l'entrada CarCode. Roman estable mentre estigui actiu el senyal DataReady.
- **TotalLaps:** nombre total de voltes de la carrera, codificat en binari natural amb 8 bits. Roman estable des de la primera activació del senyal DataReady, i no canvia al llarg de la carrera.
- **DataReady:** senyal que s'activa (lògica positiva) quan tenim disponible una nova dada d'un cotxe que ha completat una volta i el temps que ha trigat. Es manté actiu fins que el sistema activa la sortida DataOk.
- **PCI:** senyal de posada en condicions inicials del sistema.
- **T:** senyal de sincronisme del sistema.

Sortides:

- **DataOk:** senyal que activa el sistema quan ha acabat de processar la informació d'un cotxe i el seu temps de volta. Ha de romandre actiu fins que no es desactivi l'entrada DataReady.
- **EndProcess:** senyal que activa el sistema quan ja està disponible la informació dels 10 primers classificats a la FIFO. Ha de romandre actiu fins que s'apagui el sistema per extreure la FIFO.

Funcionament del sistema:

Al posar en marxa el sistema, aquest roman a l'espera de l'activació de DataReady, que indica que hi ha disponibles el codi identificatiu d'un cotxe i el temps que ha trigat a fer la volta. El sistema ha de comprovar si aquest temps és menor que el millor temps de volta d'aquest cotxe emmagatzemat a la RAM i, si cal, actualitzar-lo. Tanmateix, ha d'incrementar el nombre de voltes completades per aquest cotxe. Cada cop que un cotxe hagi completat el total de voltes de la carrera, el sistema ha de guardar a la FIFO el seu codi, la posició en què ha acabat i el temps de la seva millor volta. Quan s'hagin processat les dades d'un cotxe, el sistema activarà la sortida DataOk, i romandrà a l'espera de la desactivació de l'entrada DataReady, moment en que tornarà a restar a l'espera d'una nova dada d'un altre cotxe que completi una volta. Quan s'hagin emmagatzemat a la FIFO els 10 primers cotxes que acaben la carrera es donarà per finalitzat el procés i s'activarà el senyal EndProcess, que restarà actiu fins que s'apagui el sistema per extreure la FIFO.

Podem suposar que el sistema és prou ràpid per processar correctament les dades d'un cotxe abans que el següent completi una volta.

Es demana:

- a) Dissenya el hardware necessari del sistema, que ha d'incloure la RAM i la FIFO, etiquetant convenientment els senyals d'interès i indicant amb claredat la interconnexió d'aquest hardware amb la màquina d'estats de control.
- b) Dissenya el diagrama d'estats de la màquina d'estats que controla el sistema fent servir l'estàndard UML (no cal la implementació de la màquina d'estats a nivell de portes ni biestables).
- c) A les carreres organitzades per la FIA participen al voltant d'una trentena de cotxes com a màxim. Fes un càlcul del percentatge de paraules d'emmagatzemament ocupades a la memòria RAM i comenta el resultat.

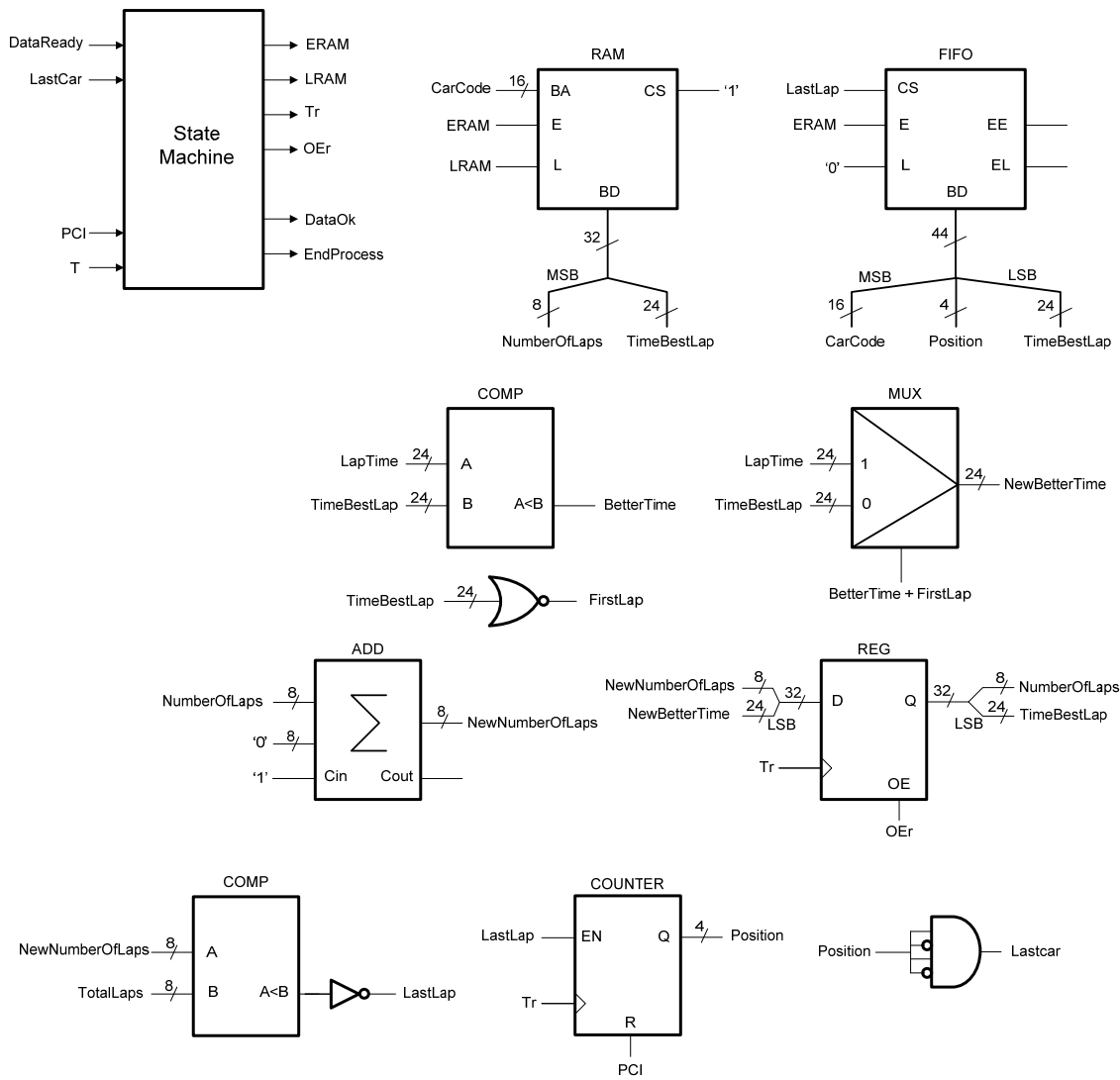
Pel disseny del hardware es pot emprar qualsevol porta, bloc combinacional, biestable, registre, comptador o memòria "estàndard" estudiat a classe.

Solució:

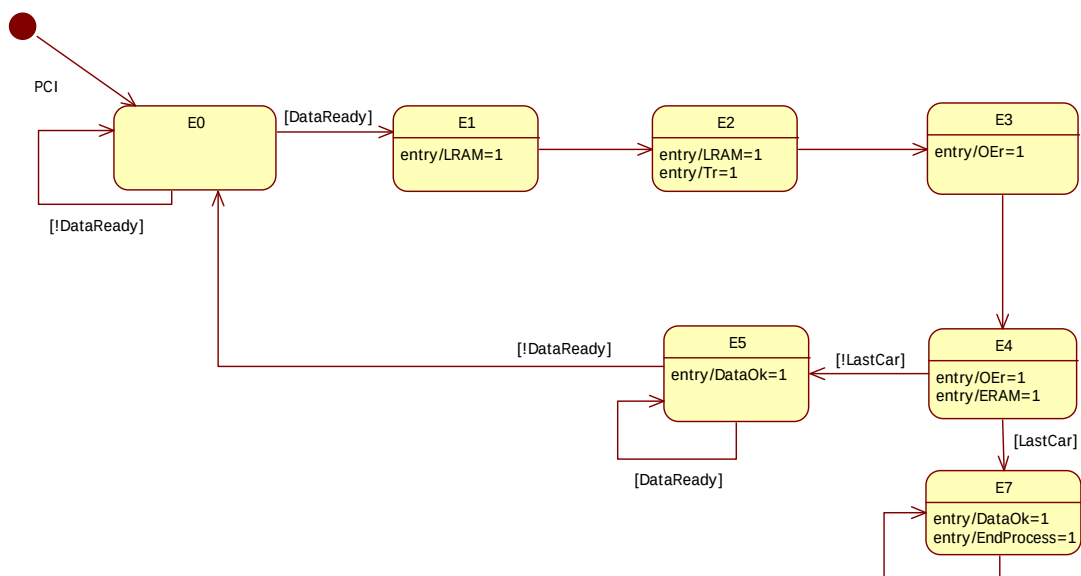
- a) En ser un problema de disseny no existeix una única solució. Aquí es presenta un hardware que realitza certes tasques que també es podrien realitzar directament des de la màquina d'estats. Convé recordar que el diagrama d'estats que es presenta a l'apartat b) va lligat a aquest hardware concret. Altres propostes de hardware necessitaran d'una màquina d'estats diferent.

Els elements de hardware necessaris per a la resolució del problema són:

- Les memòries RAM i FIFO, amb indicació correcta de la seva estructura de busos d'adreces, de dades, senyals de control i les seves connexions.
- Detecció de si és la primera volta (TimeBestLap tot a zeros).
- Detecció de si es millora el temps de volta (per exemple, un comparador).
- Selecció del nou temps de volta a actualitzar a la RAM (per exemple, un multiplexor controlat per la sortida del comparador anterior).
- Bloc que incrementa en una la quantitat de voltes per actualitzar el valor a la RAM (per exemple, un sumador).
- Registres per emmagatzemar la informació a actualitzar a la RAM (si no es guarden en registres, es perden al desactivar la lectura de la RAM).
- Detecció de si un cotxe finalitza la carrera i generador incremental de la posició en què ha acabat (per exemple, un comparador amb TotalLaps i un comptador per la posició).
- Detecció de la finalització del procés quan han acabat 10 cotxes (per exemple, un comparador de la sortida del comptador de posició amb "10").
- Bloc representatiu de la màquina d'estats amb totes les seves entrades i sortides.



b)



c) $(30/2^{16}) \times 100 \approx 0.05\%$, el que suposa un desaprofitament total d'espai a la RAM. Seria molt millor una CAM de 30 o 40 paraules de capacitat indexada per CarCode.